960121

Interactive UI with JS

Session 03

Objective

- **Interactive UI with JS**, we shift our focus from just *fetching* data to *manipulating* it based on user intent.
- While Session 2 focused on data acquisition, Session 3 focuses on **state and reaction**.
- A professional application must respect the user's time and cognitive load. If a user wants "Shoes," showing them "Toasters" is a failure of business logic.

HTML vs UI

- **HTML** (**H**yper**T**ext **M**arkup **L**anguage): This is the **structure**. It is a coding language used to define the elements on a page—headings, paragraphs, buttons, and links. It tells the browser what content exists.

- **UI** (**U**ser **I**nterface): This is the **experience**. It is the visual result of that structure once styling (CSS), behavior (JavaScript), and design principles are applied. It is what the user actually sees, touches, and interacts with.

HTML vs UI

- In web apps, HTML serves as the **UI skeleton**. You cannot have a web UI without HTML.
  - **The Foundation**: Every button you click or form you fill out in a web app starts as an HTML tag (e.g., `<button>` or `<input>`).
- **The Bridge**: UI designers create the "look and feel" in tools like Figma. Developers then use HTML to translate those visual designs into a functional format that a web browser can understand.
- **The Hierarchy**: HTML provides the semantic order (what comes first, what is grouped together), which is the most critical part of **UI Accessibility**.

HTML vs DOM

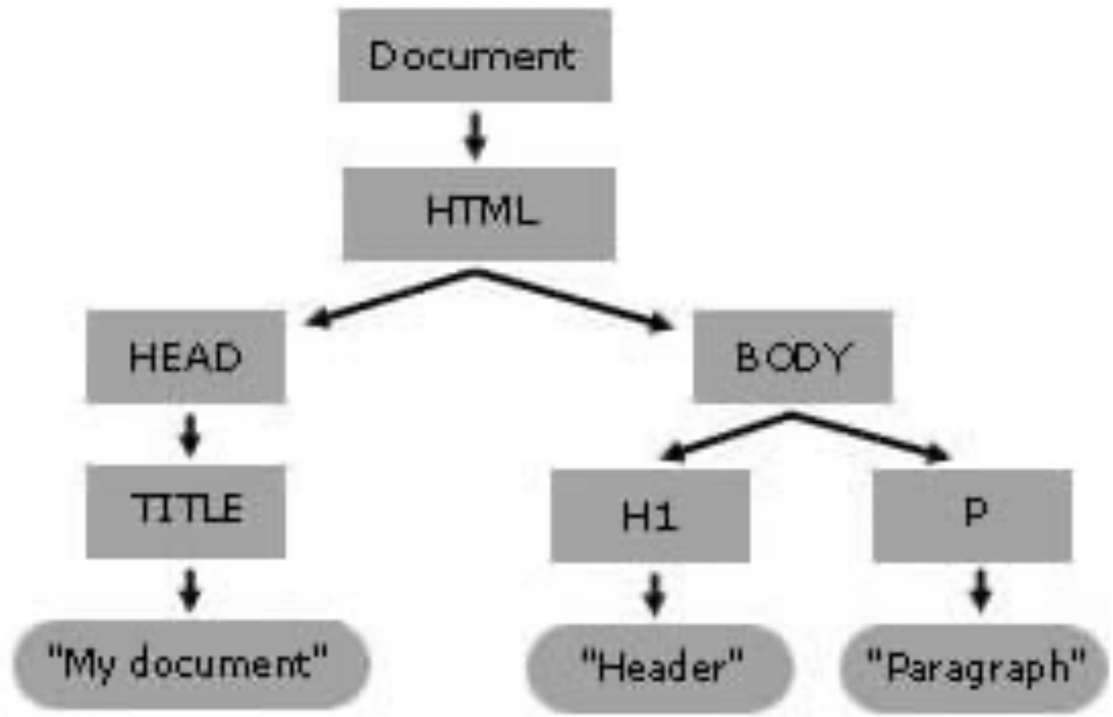
- **HTML** and the **DOM** are closely related, they represent the same content in two different states: **static** vs. **dynamic**.
- **DOM (Document Object Model):** This is a **living representation**. Once the browser receives the HTML, it parses it into an object-oriented "tree" structure that stays in the computer's memory.

DOM

- DOM is a programming interface (**API**) for web (HTML or XML) documents that presents them as a structured tree of objects.
- It represents the webpage (UI) so that programs like JavaScript can change the document structure, style, and content.
- The DOM represents a document as a tree of nodes and objects; this way, programming languages can interact with the webpage (UI).

DOM tree

```
<html lang="en">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <h1>Header</h1>
    <p>Paragraph</p>
  </body>
</html>
```



https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

DOM vs JavaScript

- **DOM** is *written* in JavaScript, but *uses* the DOM to access the document and its elements.
- All HTML tags and other elements in an HTML document are parts of the DOM for that document.
- HTML tags and other elements **can be accessed and manipulated** using the DOM and a scripting language such as JavaScript.

Accessing the DOM

```
<html lang="en">
  <head><title>Accessing DOM</title></head>
  <body>
    <script>
      // create elements in an otherwise empty HTML page
      const heading = document.createElement("h1");
      const headingText = document.createTextNode("Big Head!");
      heading.appendChild(headingText);
      document.body.appendChild(heading);
    </script>
  </body>
</html>
```

Intercative UI (DOM + JS)

- HTML document → Static structure and content
- The browser reads the HTML document and then creates the DOM.
- After creating the DOM, it is “listening”.
- DOM Nodes can also have **event handlers** attached to them.
- Once an event is triggered, the event handlers get executed.

DOM Events

DOM events are signals that something has occurred in the browser, such as *a user clicking a button* or *a page finishing its load process*.

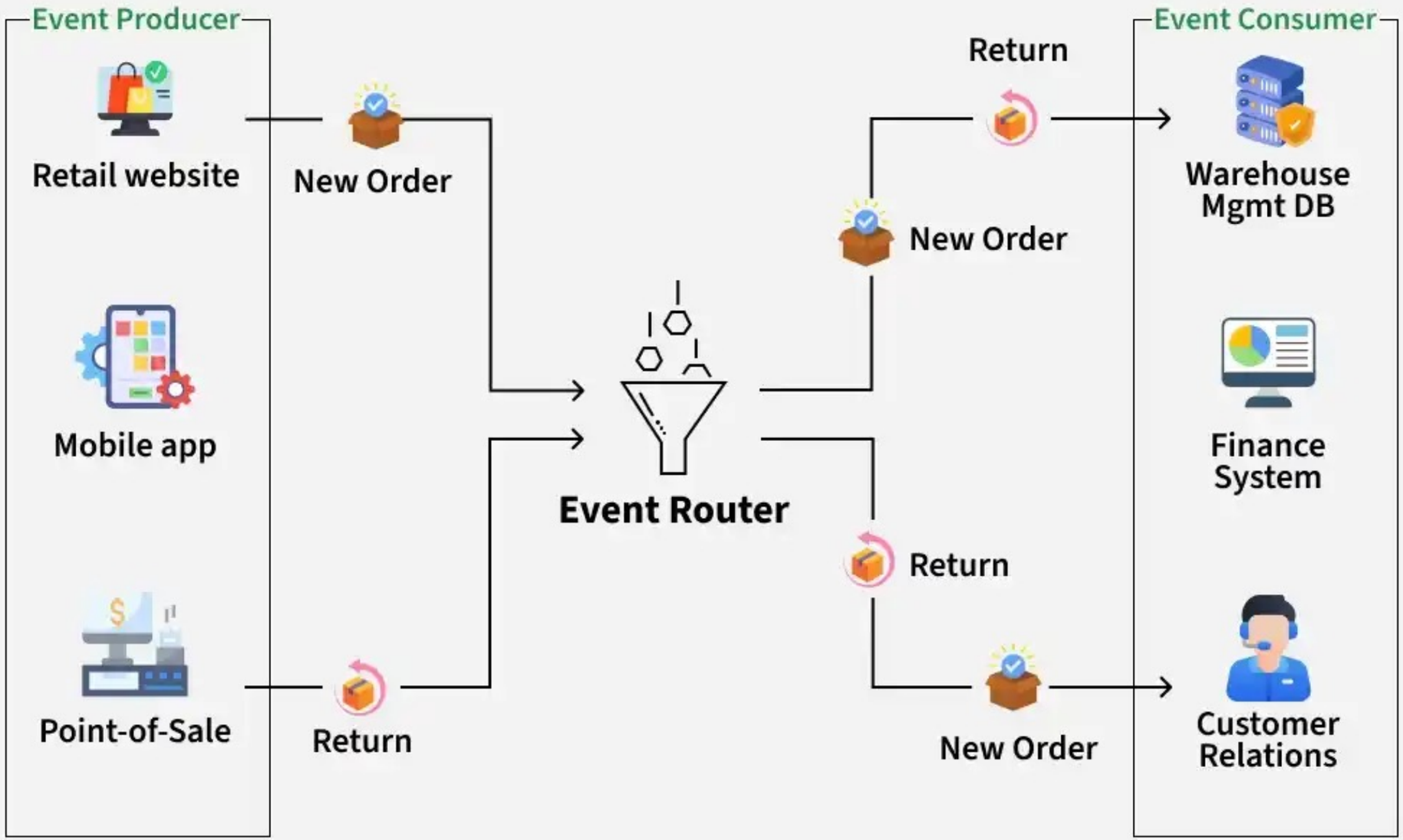
Example:

Event type	Description
Inputs	Events related to HTML input elements e.g. `<input>`, `<select>`, or `<textarea>`.
Form	Events related to forms being constructed, reset and submitted.

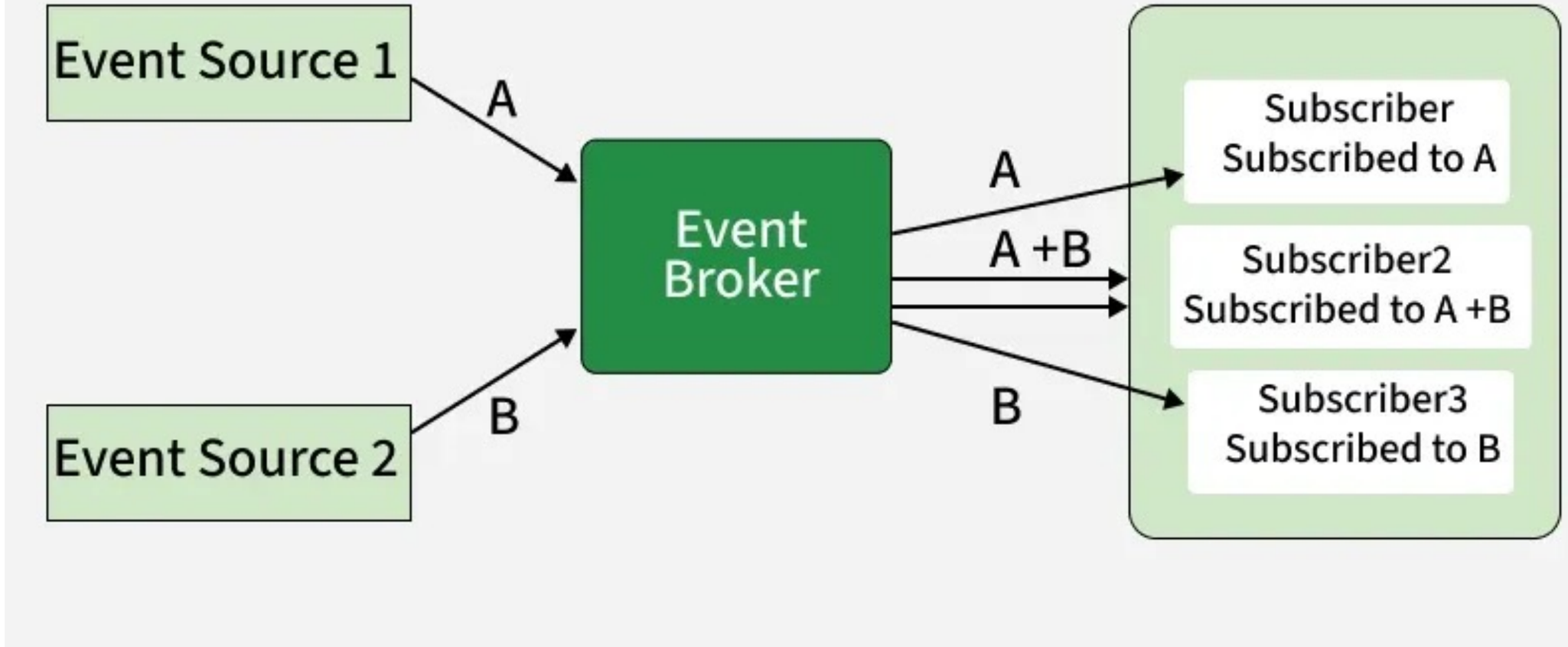
Event-Driven Architecture(EDA)

- EDA is a **software design approach** where system components communicate by **producing and responding to events**, such as user actions or system state changes.
- **Example**: *In an e-commerce system, when a customer places an order, an Order Placed event is generated. Services such as payment processing, inventory management, and email notifications don't constantly check the order system; instead, they respond independently when the event occurs.*

Event-Driven Architecture of E-Commerce Site



https://www.geeksforgeeks.org/system-design/event-driven-architecture-in-an-e-commerce-system/



https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/

EDA

Every interaction consists of:

- **The Producer (The Trigger)**: The HTML element where the action starts (e.g., a "Buy Now" button or a search input field).
- **The Event (The Message)**: The object containing information (or data) about what happened (e.g., Which key was pressed? What is the current value of the dropdown?).
- **The Consumer (The Handler)**: The JavaScript function that executes the business logic (e.g., adding an item to a cart array).

EDA

Specific mechanisms are what make the UI feel "alive":

The Event Loop:

- JavaScript is single-threaded.
  - While it waits for a user to click or a file to download, it doesn't freeze—it puts those "events" into a queue.
- **Critical Thinking Point:** If you run a heavy calculation (like processing 10,000 products) directly on the main thread, your UI "hangs."
- We can avoid this using **Asynchronous** logic

EDA

Specific mechanisms are what make the UI feel "alive":

Event Delegation:

- Instead of adding a "click" listener to 100 different product buttons (which consumes memory), we add one listener to the parent container.
- **Business Logic:** This makes the app scalable. As the product catalog grows, the memory footprint stays small.

EDA

Specific mechanisms are what make the UI feel "alive":

Debouncing (Advanced Interaction):

- If you implement a search bar that filters on every single keystroke, it might trigger 10 requests in 2 seconds.
- **Programming Logic:** We solve this with "debounce"—waiting for the user to stop typing for 300ms before running the filter. This saves server costs and improves performance.

EDA for interactive catalog

To build an interactive catalog, we need to bridge the gap between HTML elements and JavaScript logic:

- **Event Listeners (The Triggers)**: Understanding that the DOM is "listening." We'll focus on `click` for buttons and `input/change` for search bars and filters.
- **State Management (The Brain)**: Introducing the concept of a "source of truth." We don't just filter the HTML; we filter the **Data Array** and then re-render the UI.
- **High-Order Functions** (`.filter()`, `.sort()`): Moving beyond basic loops to functional programming. This is where the real "Logic" happens.

Referance

- **Document Object Model (DOM)** (https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model)
- **Event-Driven Architecture(EDA)** (https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/)

Class Activity:
The "Instant Response" Catalog

1. The Scenario:
   - **The Challenge**: A static list of 100 products is overwhelming. We will respect the user's time looking for the product they intend to buy.
   - **Goal**: We want to add a search bar and a category dropdown. When the user types or selects a category, the products must filter *instantly* without refreshing the page.

The "Instant Response" Catalog

2. (15 min.) UML:

- Draw an **Activity Diagram** for searching for a product/selecting a product category.
- **Draw the flow:** `User Action` → `Capture Input` → `Filter Array` → `Update DOM`.
- **Consider this logic**:
  - If the search box is empty AND no category is selected, what should show?
  - If the search returns no results, what does the user see?

The "Instant Response" Catalog

3. (5 min.) Logic Table:

User Action (Trigger)	DOM Event	Business Logic Action	Expected UI Change
Clicks "Price: High to Low"	change	**Sort** `allProducts` array by price	Catalog re-orders instantly
Types "Nike" in Search	input	**Filter** array for "Nike"	Non-matching products disappear

The "Instant Response" Catalog

4. **(10 min.)** Peer Evaluation:
   - Students swap drawings.
   - **Critical Thinking Question:**
     - Did your friend’s logic handle 'Case Sensitivity’?
     - If I search for 'shoes' vs 'Shoes', does it break?"

The "Instant Response" Catalog

5. (10 min.) GenAI Prompt Engineering:

- Use UML logic to craft a structured prompt.

The "Instant Response" Catalog

6. **(5 min.)** Integration:
   - Open your one-page template in VS Code.
   - Explore your HTML and identify the “search” input box and the “category” dropdown list. If there are none, locate where we should place them.
   - Bind the AI-generated function to the `input` and `change` event listener.
   - Reuse the `renderProducts()` function from Session 2 to display the new filtered list.

The "Instant Response" Catalog

7. (10 min.) Debugging & Testing:

- Open your web on localhost via VS Code (Live Server).
- If an error is found, use the Browser DevTools to debug the code.

- Test "**Edge Case**" case: type a special character (like `@` or `#`) or a string that doesn't exist.

The "Instant Response" Catalog

8. (5 min.) Version Control:

- Terminal Command:
  - `git add`
  - `git commit -m "feat: add real-time search and category filtering."`
  - `git push.`
- VS Code:
- GitHub Desktop:

Key Takeaway

1. The UI is a Reflection of Data (State)

- **The Lesson**: We don't "delete a row" from an HTML table; we remove an item from a JavaScript Array (The State) and tell the UI to re-render.

- **Why**: This makes the application predictable. If the data is correct, the UI will be correct.

Key Takeaway

2. Decoupling Logic from Triggers

- **The Lesson:** A `filterProducts()` function shouldn't care if it was triggered by an input box, a dropdown, or other DOM nodes (HTML elements).

- **Why:** This allows for Reusability. It makes the code easier to test and allows GenAI to write cleaner, modular functions rather than "spaghetti code" where everything is tangled together.

Key Takeaway

3. Anticipating the User (UX as Logic)

- **The Lesson**: Event-driven programming isn't just about successful clicks; it’s about handling the "silence." If a search returns zero results, the business logic must dictate a "No results found" state.

- **Why**: This builds **Critical Thinking**. An architect must account for "Edge Cases" (empty searches, invalid inputs) just as much as the "Happy Path."